

CSC467 Technical Analysis

December 8, 2024

```
[1]: import os
import sys

os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-17-openjdk-amd64/"
os.environ["SPARK_HOME"] = "/usr/local/spark/"
spark_path = os.environ['SPARK_HOME']
sys.path.append(spark_path + "/bin")
sys.path.append(spark_path + "/python")
sys.path.append(spark_path + "/python/pyspark/")
sys.path.append(spark_path + "/python/lib")
sys.path.append(spark_path + "/python/lib/pyspark.zip")
sys.path.append(spark_path + "/python/lib/py4j-0.10.9.7-src.zip")

number_cores = 8
memory_gb = 16
```

```
[2]: import findspark
import pyspark
import json

findspark.init()
conf = (pyspark.SparkConf().setMaster('local[{}]'.format(number_cores)).
    ↪set('spark.driver.memory', '{}g'.format(memory_gb)))

sc = pyspark.SparkContext(conf=conf)
```

```
[3]: reviewRDD = sc.textFile("data/yelp_academic_dataset_review.json")
userRDD = sc.textFile("data/yelp_academic_dataset_user.json")
businessRDD = sc.textFile("data/yelp_academic_dataset_business.json")

reviewFieldsRDD = reviewRDD.map(lambda line: json.loads(line)).map(lambda r: ↵
    ↪(r['review_id'], r['business_id'], r['user_id'], r['stars'], r['text']))
userFieldsRDD = userRDD.map(lambda line: json.loads(line)).map(lambda u: ↵
    ↪(u['user_id'], u['name'], u['yelping_since'], u['friends']))
businessFieldsRDD = businessRDD.map(lambda line: json.loads(line)).map(lambda b: ↵
    ↪(b['business_id'], b['name'], b['review_count'], b['stars']))
```

```

reviewFieldsSamplePath = "output/desAnalysis/reviewFieldsSample"
reviewFieldsSample = sc.parallelize(reviewFieldsRDD.map(json.dumps).take(3))
reviewFieldsSample.coalesce(1).saveAsTextFile(reviewFieldsSamplePath)
print(f"Review Fields Sample saved to {reviewFieldsSamplePath}")

userFieldsSamplePath = "output/desAnalysis/userFieldsSample"
userFieldsSample = sc.parallelize(userFieldsRDD.map(json.dumps).take(3))
userFieldsSample.coalesce(1).saveAsTextFile(userFieldsSamplePath)
print(f"User Fields Sample saved to {userFieldsSamplePath}")

businessFieldsSamplePath = "output/desAnalysis/businessFieldsSample"
businessFieldsSample = sc.parallelize(businessFieldsRDD.map(json.dumps).take(3))
businessFieldsSample.coalesce(1).saveAsTextFile(businessFieldsSamplePath)
print(f"Business Fields Sample saved to {businessFieldsSamplePath}")

```

Review Fields Sample saved to output/desAnalysis/reviewFieldsSample
 User Fields Sample saved to output/desAnalysis/userFieldsSample
 Business Fields Sample saved to output/desAnalysis/businessFieldsSample

```

[5]: # Filter reviews with 1 or 2 stars
lowRatedReviewsRDD = reviewFieldsRDD.filter(lambda r: r[3] in [1, 2])

lowRatedReviewsSamplePath = "output/desAnalysis/lowRatedReviewsSample"
lowRatedReviewsSample = sc.parallelize(lowRatedReviewsRDD.map(json.dumps).
    ↪take(3))
lowRatedReviewsSample.coalesce(1).saveAsTextFile(lowRatedReviewsSamplePath)
print(f"Low Rated Reviews Sample saved to {lowRatedReviewsSamplePath}")

```

Low Rated Reviews Sample saved to output/desAnalysis/lowRatedReviewsSample

```

[6]: lowRatedReviewsByUserRDD = lowRatedReviewsRDD.map(lambda r: (r[2], r))
usersByID = userFieldsRDD.map(lambda u: (u[0], u))
lowRatedReviewsWithUsersRDD = lowRatedReviewsByUserRDD.join(usersByID)
pairedLowReviewsRDD = lowRatedReviewsWithUsersRDD.map(lambda x: (x[0], x[1][0], ↪
    ↪x[1][1]))

```

```

[7]: userFriendsRDD = userFieldsRDD.map(lambda u: (u[0], set(u[3].split(', '))))
lowRatedWithFriendsRDD = lowRatedReviewsByUserRDD.join(userFriendsRDD)
lowRatedBusinessRDD = lowRatedWithFriendsRDD.map(lambda x: (x[1][0][1], (x[0], ↪
    ↪x[1][0], x[1][1])))
allReviewsByBusinessRDD = lowRatedReviewsRDD.map(lambda r: (r[1], (r[2], r)))
joinedBusinessReviewsRDD = lowRatedBusinessRDD.join(allReviewsByBusinessRDD.
    ↪groupByKey())
friendsReviewedRDD = joinedBusinessReviewsRDD.map(lambda x: (x[1][0][0], x[0], ↪
    ↪x[1][0][1][0], [(friendReview[0], friendReview[1][0]) for friendReview in ↪
    ↪x[1][1] if friendReview[0] in x[1][0][2]])).filter(lambda x: len(x[3]) > 0)
groupedByUserRDD = friendsReviewedRDD.map(lambda x: (x[0], {'businessId': x[1], ↪
    ↪'userReviewId': x[2], 'friendsReviews': x[3]})).groupByKey()

```

```
multipleBusinessesRDD = groupedByUserRDD.filter(lambda x: len(x[1]) > 1)
finalResultRDD = multipleBusinessesRDD.map(lambda x: {'userId': x[0],
↳ 'businessReviews': list(x[1])})
```

```
[8]: finalResultRDD = finalResultRDD.persist()

finalResultSamplePath = "output/desAnalysis/finalResultSample"
finalResultSample = sc.parallelize(finalResultRDD.map(json.dumps).take(3))
finalResultSample.coalesce(1).saveAsTextFile(finalResultSamplePath)
print(f"Final Result Sample saved to {finalResultSamplePath}")
```

Final Result Sample saved to output/desAnalysis/finalResultSample

```
[9]: jsondump = finalResultRDD.map(json.dumps)
new = jsondump.coalesce(1)
outputDirectory = "output/desAnalysis/final"
new.saveAsTextFile(outputDirectory)
print(f"Saved to {outputDirectory}")
```

Saved to output/desAnalysis/final

```
[10]: suspectReviewerRDD = sc.textFile("output/desAnalysis/final/part-00000").
↳ map(json.loads)
suspectReviewerRDD.take(1)
```

```
[10]: [{'userId': 'cRANsQ5E_sxeQ0Aoop3drg',
'businessReviews': [{'businessId': '5j9TfW8RrdkpaHlaBwt6bQ',
'userReviewId': 'Udlabx_W_s1U0jYNApIDg',
'friendsReviews': [['hw5zmHc2Qai0-LCIs5MqHA', 'wFwWCuRRfpwd_itwe5JNTA'],
['56oHQV84bjSMtLkIOsh_4A', 'VK6MoyM9WOkdeCS3jrxdxw'],
['bnKo6L4MgcUbLZH7fmnkrA', 'n3yCH2cjBrlgjbFDwkLCig'],
['bnKo6L4MgcUbLZH7fmnkrA', 'viRL3ynZum5V5YZMXMyQSg'],
['u0igpMz49lHLul-9yH9C0w', 'usWDFj78csqY7T4l0gSG6A'],
['CqQuA6sT1_CmNJWR3qkzeg', '_V-jDh7J90YQdZUVoGG-rA']]}],
{'businessId': 'pAFt0cz710oLQ-e_J6AIzw',
'userReviewId': 'q5lvZuoGrdNZtgBY7AHJOw',
'friendsReviews': [['YNkRUoruuV4g3jSDIyXw9A', 't66qj4cbPYAYlfpqxchYvg'],
['-vPtYuV_dMPb0Bbzg9ECVw', 'KIAqCeQuhC2BQNI-p9AXLw'],
['EmurkIplcp0kh4hUnSs_ww', 'HKege4lBV85T-flscfoXgQ']]}]}
```

```
[11]: firstUserEntry = suspectReviewerRDD.take(1)[0]
reviewIDs = set([business.get('userReviewId', None) for business in
↳ firstUserEntry.get('businessReviews', []) if business.get('userReviewId') is
↳ not None] + [friend[1] for business in firstUserEntry.get('businessReviews',
↳ []) for friend in business.get('friendsReviews', [])])
broadcastReviewIDs = sc.broadcast(reviewIDs)
filteredReviewsRDD = reviewRDD.filter(lambda line: json.
↳ loads(line)['review_id'] in broadcastReviewIDs.value)
```

```

filteredReviews = filteredReviewsRDD.collect()
filteredReviewsPath = "output/desAnalysis/filteredReviews"
filteredReviewsSave = sc.parallelize(filteredReviews)
filteredReviewsSave.coalesce(1).saveAsTextFile(filteredReviewsPath)
print(f"Filtered Reviews saved to {filteredReviewsPath}")

```

Filtered Reviews saved to output/desAnalysis/filteredReviews

```

[12]: suspectEntries = suspectReviewerRDD.take(1)
targetUserIDs = [entry.get('userId', None) for entry in suspectEntries if entry.
    ↳get('userId') is not None]
broadcastUserIDs = sc.broadcast(set(targetUserIDs))
filteredUsersRDD = userRDD.filter(lambda line: json.loads(line)['user_id'] in
    ↳broadcastUserIDs.value)
filteredUsers = filteredUsersRDD.collect()
filteredUsersPath = "output/desAnalysis/filteredUsers"
filteredUsersSave = sc.parallelize(filteredUsers)
filteredUsersSave.coalesce(1).saveAsTextFile(filteredUsersPath)
print(f"Filtered Users saved to {filteredUsersPath}")

```

Filtered Users saved to output/desAnalysis/filteredUsers

```

[13]: suspectUserIDs = [(entry.get('userId', None), entry) for entry in
    ↳suspectReviewerRDD.take(5) if entry.get('userId') is not None]
suspectUserIDSet = set([userID for userID, _ in suspectUserIDs])
broadcastUserIDs = sc.broadcast(suspectUserIDSet)
filteredUserNamesRDD = userRDD.map(json.loads).filter(lambda user:
    ↳user['user_id'] in broadcastUserIDs.value).map(lambda user:
    ↳user['user_id'], user['name']))
userNamesDict = filteredUserNamesRDD.collectAsMap()
linkedReviews = [
    {
        'userId': userID,
        'userName': userNamesDict.get(userID, "Unknown"),
        'suspectReviewData': suspectData
    }
    for userID, suspectData in suspectUserIDs
]
linkedReviewsPath = "output/desAnalysis/linkedReviews"
linkedReviewsSave = sc.parallelize(linkedReviews)
linkedReviewsSave.coalesce(1).saveAsTextFile(linkedReviewsPath)
print(f"Linked Reviews saved to {linkedReviewsPath}")

```

Linked Reviews saved to output/desAnalysis/linkedReviews

```

[14]: suspiciousFriendCountsRDD = friendsReviewedRDD.map(lambda x: (x[0], len(x[3])))
totalSuspiciousFriendCountsRDD = suspiciousFriendCountsRDD.reduceByKey(lambda
    ↳a, b: a + b)

```

```

userNamesRDD = userRDD.map(lambda line: json.loads(line)).map(lambda user:
    ↪(user['user_id'], user['name']))
linkedSuspiciousCountsRDD = totalSuspiciousFriendCountsRDD.join(userNamesRDD)
sortedSuspiciousCountsRDD = linkedSuspiciousCountsRDD.sortBy(lambda x: x[1][0],
    ↪ascending=False)
topSuspiciousReviewers = sortedSuspiciousCountsRDD.take(15)
print("Top Suspicious Reviewers:")
for userID, (totalCount, name) in topSuspiciousReviewers:
    print(f"User ID: {userID}, Name: {name}, Total Suspicious Friend Reviews:
    ↪{totalCount}")
topSuspiciousReviewersPath = "output/desAnalysis/topSuspiciousReviewers"
topSuspiciousReviewers = sc.parallelize(sortedSuspiciousCountsRDD.map(json.
    ↪dumps).take(15))
topSuspiciousReviewers.coalesce(1).saveAsTextFile(topSuspiciousReviewersPath)
print(f"\nTop Suspicious Reviewers Sample saved to
    ↪{topSuspiciousReviewersPath}")

```

Top Suspicious Reviewers:

```

User ID: _BcWyKQL16ndpBdggh2kNA, Name: Karen, Total Suspicious Friend Reviews:
1315
User ID: ET8n-r7glWYqZhuR6GcdNw, Name: Michelle, Total Suspicious Friend
Reviews: 1092
User ID: bJ5FtCtZX3ZZacz2_2PJjA, Name: Bill, Total Suspicious Friend Reviews:
475
User ID: 1HM81n6n4iPIFU5d2Lokhw, Name: Shannon, Total Suspicious Friend Reviews:
461
User ID: CfX4sTIFFNaRchNswqhVfg, Name: Christopher, Total Suspicious Friend
Reviews: 399
User ID: E4BsVQnG5zetbwv2x8QIWg, Name: Charles, Total Suspicious Friend Reviews:
395
User ID: pou3BbKsIozfH50rxmnMew, Name: Brett, Total Suspicious Friend Reviews:
367
User ID: Ase_kJIYuT6yOsqqVPuWUA, Name: Teneha, Total Suspicious Friend Reviews:
319
User ID: bYENop4BuQepBjM1-BI3fA, Name: Steven, Total Suspicious Friend Reviews:
317
User ID: D8CF2H3DYtNJlZROiITMHQ, Name: ThaDragonSourceRa ®, Total Suspicious
Friend Reviews: 305
User ID: nnImk681KaRqUVHlSfZjGQ, Name: Carlos, Total Suspicious Friend Reviews:
290
User ID: MmEYowrsA4j3K4gc9Th53g, Name: Todd, Total Suspicious Friend Reviews:
286
User ID: riWDgbauId8TK7YFVgMNJw, Name: Rene, Total Suspicious Friend Reviews:
285
User ID: VL12EhEdT4OWqGqOnIqkzw, Name: Mark, Total Suspicious Friend Reviews:
268
User ID: Xw7ZjaGfr0WNVt6s_5KZfA, Name: Marielle, Total Suspicious Friend

```

Reviews: 268

Top Suspicious Reviewers Sample saved to
output/desAnalysis/topSuspiciousReviewers

```
[15]: # est runtime ~ 7-8 minutes
userFriendsRDD = userFieldsRDD.map(lambda u: (u[0], set(u[3].split(', '))))
friendReviewsRDD = userFriendsRDD.flatMapValues(lambda friends: friends).
    ↪map(lambda x: (x[1], x[0]))
allFriendReviewsRDD = reviewFieldsRDD.map(lambda r: (r[2], 1)).
    ↪join(friendReviewsRDD)
totalFriendCountsRDD = allFriendReviewsRDD.map(lambda x: (x[1][1], x[1][0])).
    ↪reduceByKey(lambda a, b: a + b)
userReviewCountsRDD = userRDD.map(lambda line: json.loads(line)).map(lambda u:
    ↪user: (user['user_id'], user['review_count']))

# suspicious, total friend, individual review counts respectively
combinedCountsRDD = totalSuspiciousFriendCountsRDD.
    ↪fullOuterJoin(totalFriendCountsRDD).fullOuterJoin(userReviewCountsRDD).
    ↪mapValues(
        lambda counts: (
            counts[0][0] if counts[0] and counts[0][0] is not None else 0,
            counts[0][1] if counts[0] and counts[0][1] is not None else 0,
            counts[1] if counts[1] is not None else 0
        )
    )

finalUserCountsRDD = combinedCountsRDD.join(userNamesRDD).persist()
sortedFinalCountsRDD = finalUserCountsRDD.sortBy(lambda x: x[1][0][0],
    ↪ascending=False)
topUsers = sortedFinalCountsRDD.take(50)
print("Top Suspect Reviewers (suspicious, total, and individual review counts):
    ↪")
for userId, ((suspiciousCount, totalCount, reviewCount), name) in topUsers:
    print(f"User ID: {userId}, Name: {name}, Suspicious Reviews:
    ↪{suspiciousCount}, Total Friend Reviews: {totalCount}, Individual Reviews:
    ↪{reviewCount}")
topUsersPath = "output/desAnalysis/topUsers"
topUsers = sc.parallelize(sortedFinalCountsRDD.map(json.dumps).take(50))
topUsers.coalesce(1).saveAsTextFile(topUsersPath)
print(f"Top Users Sample saved to {topUsersPath}")
```

Top Suspect Reviewers (suspicious, total, and individual review counts):
User ID: _BcWyKQL16ndpBdggh2kNA, Name: Karen, Suspicious Reviews: 1315, Total
Friend Reviews: 146544, Individual Reviews: 4274
User ID: ET8n-r7glWYqZhuR6GcdNw, Name: Michelle, Suspicious Reviews: 1092, Total
Friend Reviews: 381612, Individual Reviews: 2256

User ID: bJ5FtCtZX3ZZacz2_2PJJA, Name: Bill, Suspicious Reviews: 475, Total Friend Reviews: 101598, Individual Reviews: 1512

User ID: 1HM81n6n4iPIFU5d2Lokhw, Name: Shannon, Suspicious Reviews: 461, Total Friend Reviews: 32947, Individual Reviews: 2317

User ID: CfX4sTIFFNArchNswqhVfg, Name: Christopher, Suspicious Reviews: 399, Total Friend Reviews: 37223, Individual Reviews: 1405

User ID: E4BsVQnG5zetbwv2x8QIWg, Name: Charles, Suspicious Reviews: 395, Total Friend Reviews: 34817, Individual Reviews: 726

User ID: pou3BbKsIozfH50rxmnMew, Name: Brett, Suspicious Reviews: 367, Total Friend Reviews: 204627, Individual Reviews: 2812

User ID: Ase_kJIYuT6yOsqqVPuWUA, Name: Teneha, Suspicious Reviews: 319, Total Friend Reviews: 71234, Individual Reviews: 1134

User ID: bYENop4BuQepBjM1-BI3fA, Name: Steven, Suspicious Reviews: 317, Total Friend Reviews: 82785, Individual Reviews: 1562

User ID: D8CF2H3DYtNJlZROiITMHQ, Name: ThaDragonSourceRa ®, Suspicious Reviews: 305, Total Friend Reviews: 19989, Individual Reviews: 323

User ID: nnImk681KaRqUVHlSfZjGQ, Name: Carlos, Suspicious Reviews: 290, Total Friend Reviews: 105173, Individual Reviews: 1107

User ID: MmEYowrsA4j3K4gc9Th53g, Name: Todd, Suspicious Reviews: 286, Total Friend Reviews: 100910, Individual Reviews: 832

User ID: riWDgbauId8TK7YFVgMNJw, Name: Rene, Suspicious Reviews: 285, Total Friend Reviews: 31993, Individual Reviews: 810

User ID: Xw7ZjaGfr0WNVt6s_5KZfA, Name: Marielle, Suspicious Reviews: 268, Total Friend Reviews: 56559, Individual Reviews: 2272

User ID: VL12EhEdT40WqGqOnIqkzw, Name: Mark, Suspicious Reviews: 268, Total Friend Reviews: 48871, Individual Reviews: 1646

User ID: Oilqbcz2m2SnwUeztGYcnQ, Name: Steven, Suspicious Reviews: 261, Total Friend Reviews: 368842, Individual Reviews: 1371

User ID: EP8o8JnQSI0WEkeodwA81A, Name: Bobby, Suspicious Reviews: 245, Total Friend Reviews: 24307, Individual Reviews: 613

User ID: qjfMBIZpQT9DDtw_BWCopQ, Name: Rachel, Suspicious Reviews: 241, Total Friend Reviews: 84680, Individual Reviews: 2331

User ID: lMY8NBPyzlPbbu-KBYfd9A, Name: Christy, Suspicious Reviews: 240, Total Friend Reviews: 133342, Individual Reviews: 1444

User ID: 5tXRxr4T24Awl7vjyCvIcQ, Name: Morgan, Suspicious Reviews: 239, Total Friend Reviews: 131397, Individual Reviews: 741

User ID: 6s-g2vFu120emhiK3FJuOQ, Name: Dave, Suspicious Reviews: 223, Total Friend Reviews: 86061, Individual Reviews: 2388

User ID: eTvp_hYnsrI5-ow_sQ31_g, Name: Patt, Suspicious Reviews: 204, Total Friend Reviews: 47193, Individual Reviews: 1028

User ID: vHc-UrI9yfl_pnnc6nJtyQ, Name: Michael, Suspicious Reviews: 200, Total Friend Reviews: 156258, Individual Reviews: 1135

User ID: y3Wtxlp0vTiqsJRkjceXiw, Name: Catherine, Suspicious Reviews: 199, Total Friend Reviews: 45040, Individual Reviews: 371

User ID: -G7Zkl1wIWBBmDOKRy_sCw, Name: Gerald, Suspicious Reviews: 197, Total Friend Reviews: 87497, Individual Reviews: 3131

User ID: 61FNJUIdQXReViksOYiILQ, Name: Michael, Suspicious Reviews: 196, Total Friend Reviews: 73375, Individual Reviews: 1020

User ID: I2XpWCHAom1JRyHXZQrnfg, Name: Kathy, Suspicious Reviews: 188, Total Friend Reviews: 42682, Individual Reviews: 1383

User ID: fEtWwhNKSoTqIWTSEwGIvg, Name: Tanya, Suspicious Reviews: 185, Total Friend Reviews: 115931, Individual Reviews: 189

User ID: y8aWX0imQ9ZgUgZ6q--nCQ, Name: Carrie, Suspicious Reviews: 175, Total Friend Reviews: 55352, Individual Reviews: 472

User ID: AHRrG3T1gJpHvtpZ-KOG_g, Name: Brittany, Suspicious Reviews: 170, Total Friend Reviews: 174454, Individual Reviews: 2492

User ID: m-n_AHwndmHAgZIsu52kYA, Name: Tracy, Suspicious Reviews: 165, Total Friend Reviews: 31023, Individual Reviews: 689

User ID: WwulXySQN8t2hwqH_yWurA, Name: Niki, Suspicious Reviews: 163, Total Friend Reviews: 194373, Individual Reviews: 1512

User ID: -kLVfaJyt0JY2-QdQoCcNQ, Name: Christina, Suspicious Reviews: 162, Total Friend Reviews: 191657, Individual Reviews: 1454

User ID: r9SOVYrdXJrdhfr7OXj8tA, Name: Dana, Suspicious Reviews: 157, Total Friend Reviews: 97231, Individual Reviews: 748

User ID: 8VNmuYmFboXleIFf-stnFg, Name: Todd, Suspicious Reviews: 156, Total Friend Reviews: 27089, Individual Reviews: 487

User ID: 5Y5KbsI5buMcNh2hTFaRpA, Name: Marcia, Suspicious Reviews: 153, Total Friend Reviews: 93349, Individual Reviews: 798

User ID: 60bFF8-uKn0AlXuSH4TlyQ, Name: Isabella, Suspicious Reviews: 151, Total Friend Reviews: 103198, Individual Reviews: 1245

User ID: GfI-d9mQePFA2PvhAd4WGQ, Name: Robert, Suspicious Reviews: 151, Total Friend Reviews: 93561, Individual Reviews: 746

User ID: 0MJ5sKX5uq7Ma5hbl4l3BQ, Name: Jenn, Suspicious Reviews: 150, Total Friend Reviews: 44235, Individual Reviews: 632

User ID: lAW03ccfMEuRZiDqvVgcOw, Name: Kristin, Suspicious Reviews: 149, Total Friend Reviews: 38602, Individual Reviews: 1578

User ID: B-s-8VUnuBjGTP3d01jsyw, Name: Allie, Suspicious Reviews: 146, Total Friend Reviews: 32800, Individual Reviews: 2505

User ID: FlXBpK_YZxLo27jcMdII1w, Name: Mallory, Suspicious Reviews: 143, Total Friend Reviews: 49046, Individual Reviews: 1158

User ID: JUT0U3HTSB3kz9Wh7N0GqA, Name: Corey, Suspicious Reviews: 141, Total Friend Reviews: 55352, Individual Reviews: 865

User ID: h8_Mzh_ndxFwxQgLZsRs9w, Name: Scott, Suspicious Reviews: 138, Total Friend Reviews: 50614, Individual Reviews: 131

User ID: lYQk0R6sPfo3WeX-l_5BuA, Name: Christopher, Suspicious Reviews: 130, Total Friend Reviews: 11869, Individual Reviews: 1396

User ID: 47oIw-CN1guUKx2_vUdsgw, Name: Stephanie, Suspicious Reviews: 129, Total Friend Reviews: 27439, Individual Reviews: 706

User ID: dqucQwA-CHWhUczwqIxKKg, Name: Dee, Suspicious Reviews: 129, Total Friend Reviews: 153688, Individual Reviews: 956

User ID: N9hqTPQu2bmI8c2Pj8F1ww, Name: Sar, Suspicious Reviews: 128, Total Friend Reviews: 106825, Individual Reviews: 844

User ID: kTpsjDUhl9BlRl0FVuNreA, Name: Alisha, Suspicious Reviews: 126, Total Friend Reviews: 159291, Individual Reviews: 378

User ID: Hn802RQijYIVLFNF5VPWTA, Name: Ali, Suspicious Reviews: 124, Total Friend Reviews: 19585, Individual Reviews: 499

Top Users Sample saved to output/desAnalysis/topUsers

```
[24]: baskets = suspectReviewerRDD.flatMap(lambda x: [
    list(set(
        [x.get('userId', None)] +
        [friend[0] for review in x.get('businessReviews', []) for friend in
        ↪review.get('friendsReviews', [])]
    )) if x.get('userId') is not None else []
])
s = 8
itemCounts = baskets.flatMap(lambda basket: [(userId, 1) for userId in basket]).
    ↪reduceByKey(lambda x, y: x + y)
freqItems = itemCounts.filter(lambda x: x[1] >= s)
freqItemLists = sc.broadcast(freqItems.map(lambda x: x[0]).collect())

numFrequentItems = freqItems.count()
print("Number of frequent items:", numFrequentItems)

pairSets = baskets.flatMap(
    lambda basket: [
        (i, j)
        for i in basket if i in freqItemLists.value
        for j in basket if j > i and j in freqItemLists.value
    ]
)
pairCounts = pairSets.map(lambda pair: (pair, 1)).reduceByKey(lambda x, y: x +
    ↪y)
freqPairs = pairCounts.filter(lambda x: x[1] >= s)
frequentPairsList = freqPairs.map(lambda x: x[0]).collect()
freqPairLists = sc.broadcast(set(frequentPairsList))
sortedFreqPairs = freqPairs.sortBy(lambda x: -x[1])

topFrequentPairs = sortedFreqPairs.take(15)
print("\nFrequent Pairs (Sorted by Count):")
for pair, count in topFrequentPairs:
    print(f"{pair}: {count}")

# Triplet generation
tripSets = baskets.flatMap(
    lambda basket: [
        (i, j, k)
        for i in basket if i in freqItemLists.value
        for j in basket if j > i and (i, j) in freqPairLists.value
        for k in basket if k > j and (j, k) in freqPairLists.value
    ]
)
```

```

tripCounts = tripSets.map(lambda triplet: (triplet, 1)).reduceByKey(lambda x, y:
    ↪ x + y)
freqTriplets = tripCounts.filter(lambda x: x[1] >= s)
sortedFreqTriplets = freqTriplets.sortBy(lambda x: -x[1])
topTriplets = sortedFreqTriplets.take(15)
print("\nFrequent Triplets (Sorted by Count):")
for triplet, count in topTriplets:
    print(f"{triplet}: {count}")

# Confidence calculations
keyPairs = freqPairs.flatMap(lambda x: [
    (x[0][0], (x[0], x[1])),
    (x[0][1], ((x[0][1], x[0][0]), x[1]))
])
pairItemSupp = keyPairs.join(itemCounts)
pairConfidence = pairItemSupp.map(lambda x: (x[1][0][0], x[1][0][1] / x[1][1]))

topPairs = pairConfidence.sortBy(lambda x: -x[1]).take(15)
print("\nTop Frequent Pairs with Confidence:")
for pair, confidence in topPairs:
    print(f"{pair}, Confidence: {confidence:.5f}")

# Confidence calculations
tripCountsKeyed = freqTriplets.flatMap(lambda x: [
    (x[0][0], (x[0], x[1])),
    (x[0][1], ((x[0][1], x[0][0], x[0][2]), x[1])),
    (x[0][2], ((x[0][2], x[0][0], x[0][1]), x[1]))
])
tripItemSupp = tripCountsKeyed.join(itemCounts)
tripConfidence = tripItemSupp.map(lambda x: (x[1][0][0], x[1][0][1] / x[1][1]))

topTripletsWithConf = tripConfidence.sortBy(lambda x: -x[1]).take(15)
print("\nTop Frequent Triplets with Confidence:")
for triplet, confidence in topTripletsWithConf:
    print(f"{triplet}, Confidence: {confidence:.5f}")

```

Number of frequent items: 1446

Frequent Pairs (Sorted by Count):

```

('ET8n-r7glWYqZhuR6GcdNw', '_BcWyKQL16ndpBdggh2kNA'): 177
('_BcWyKQL16ndpBdggh2kNA', 'bJ5FtCtZX3ZZacz2_2PJjA'): 112
('ET8n-r7glWYqZhuR6GcdNw', 'bJ5FtCtZX3ZZacz2_2PJjA'): 86
('6s-g2vFu120emhiK3FJu0Q', '_BcWyKQL16ndpBdggh2kNA'): 69
('6s-g2vFu120emhiK3FJu0Q', 'bJ5FtCtZX3ZZacz2_2PJjA'): 59
('_BcWyKQL16ndpBdggh2kNA', 'y3Wtx1p0vTiqsJRkjcexiw'): 57
('pou3BbKsIozfH50rxmnMew', 'qjfMBIZpQT9DDtw_BWCopQ'): 55
('61FNJUiDqXReViks0YiILQ', 'ET8n-r7glWYqZhuR6GcdNw'): 54
('Ase_kJIYuT6y0sqqVPuWUA', 'CfX4sTIFFNaRchNsqhVfg'): 54

```

```
( 'bYENop4BuQepBjM1-BI3fA', 'lMY8NBPpyz1Pbbu-KBYfD9A'): 53
( 'bYENop4BuQepBjM1-BI3fA', 'eTvp_hYnsrI5-ow_sQ31_g'): 53
( '6s-g2vFu120emhiK3FJu0Q', 'ET8n-r7glWYqZhuR6GcdNw'): 51
( '0i1qbcz2m2SnwUeztGYcnQ', '_BcWyKQL16ndpBdggh2kNA'): 51
( '0MJ5sKX5uq7Ma5hbl4l3BQ', '_BcWyKQL16ndpBdggh2kNA'): 51
( '_BcWyKQL16ndpBdggh2kNA', 'y8aWX0imQ9ZgUgZ6q--nCQ'): 50
```

Frequent Triplets (Sorted by Count):

```
( 'ET8n-r7glWYqZhuR6GcdNw', '_BcWyKQL16ndpBdggh2kNA', 'bJ5FtCtZX3ZZacz2_2PJjA'):
58
( '6s-g2vFu120emhiK3FJu0Q', '_BcWyKQL16ndpBdggh2kNA', 'bJ5FtCtZX3ZZacz2_2PJjA'):
39
( 'Ase_kJIYuT6y0sqqVPuWUA', 'CfX4sTIFFNArChNswqhVfg', 'E4BsVQnG5zetbwv2x8QIWg'):
37
( '6s-g2vFu120emhiK3FJu0Q', 'ET8n-r7glWYqZhuR6GcdNw', '_BcWyKQL16ndpBdggh2kNA'):
35
( 'ET8n-r7glWYqZhuR6GcdNw', '_BcWyKQL16ndpBdggh2kNA', 'y3Wtx1p0vTiqsJRkjceXiw'):
34
( '_BcWyKQL16ndpBdggh2kNA', 'bJ5FtCtZX3ZZacz2_2PJjA', 'y3Wtx1p0vTiqsJRkjceXiw'):
32
( 'ZCVEZmTB_StK-UXA3WPV1Q', '_BcWyKQL16ndpBdggh2kNA', 'bJ5FtCtZX3ZZacz2_2PJjA'):
31
( '0MJ5sKX5uq7Ma5hbl4l3BQ', '_BcWyKQL16ndpBdggh2kNA', 'bJ5FtCtZX3ZZacz2_2PJjA'):
31
( '61FNJUiDqXReViks0YiILQ', 'ET8n-r7glWYqZhuR6GcdNw', '_BcWyKQL16ndpBdggh2kNA'):
30
( '1HM81n6n4iPIFU5d2Lokhw', 'Ase_kJIYuT6y0sqqVPuWUA', 'CfX4sTIFFNArChNswqhVfg'):
30
( '1HM81n6n4iPIFU5d2Lokhw', 'CfX4sTIFFNArChNswqhVfg', 'E4BsVQnG5zetbwv2x8QIWg'):
30
( 'ET8n-r7glWYqZhuR6GcdNw', '0i1qbcz2m2SnwUeztGYcnQ', '_BcWyKQL16ndpBdggh2kNA'):
29
( 'ET8n-r7glWYqZhuR6GcdNw', 'N9hqTPQu2bmI8c2Pj8F1ww', '_BcWyKQL16ndpBdggh2kNA'):
29
( '1HM81n6n4iPIFU5d2Lokhw', 'Ase_kJIYuT6y0sqqVPuWUA', 'E4BsVQnG5zetbwv2x8QIWg'):
29
( '0MJ5sKX5uq7Ma5hbl4l3BQ', 'ET8n-r7glWYqZhuR6GcdNw', '_BcWyKQL16ndpBdggh2kNA'):
29
```

Top Frequent Pairs with Confidence:

```
( 'wcv6ypJ7oNdfBA6QiB0pgw', 'WwulXySQN8t2hwqH_yWurA'), Confidence: 1.00000
( '_xrlZRAGpfgDpVydY5D1IQ', 'CfX4sTIFFNArChNswqhVfg'), Confidence: 1.00000
( 'QwY_IrZ7Xs4N1ks06GUB0A', 'ci7Q6NvXQ8UZrBhusVX18g'), Confidence: 1.00000
( '4kjVKQhMJkdlasEkn_Nkxw', '_BcWyKQL16ndpBdggh2kNA'), Confidence: 1.00000
( 'dT60_rV9DWYS-zHXhA6S6w', 'lMY8NBPpyz1Pbbu-KBYfD9A'), Confidence: 1.00000
( 'w6RF0Qf3tL2v6q93NYmSdQ', '_BcWyKQL16ndpBdggh2kNA'), Confidence: 1.00000
( '0ikReUVUjKPtxajKx494ug', 'bYENop4BuQepBjM1-BI3fA'), Confidence: 1.00000
( 'rJZuQtzwh4S_XTMgh0ZG3w', '_BcWyKQL16ndpBdggh2kNA'), Confidence: 1.00000
```

('JITt4DBC550bSK3rNB1EOQ', 'lMY8NBPyzlPbbu-KBYfD9A'), Confidence: 1.00000
 ('JITt4DBC550bSK3rNB1EOQ', '60RKNAZwY5EUooJf624aKw'), Confidence: 1.00000
 ('N1ZCuhNOEFevaIfn6uRTNQ', 'ci7Q6NvXQ8UZrBHusVX18g'), Confidence: 1.00000
 ('8IT6b1Y0lu6BiQj1Ztj_7w', 'bJ5FtCtZX3ZZacz2_2PJjA'), Confidence: 1.00000
 ('1Ko1CajA-khSe60ZH4AtDA', '_BcWyKQL16ndpBdggh2kNA'), Confidence: 1.00000
 ('E4ptx5HC4y-XNdWxgJMZeQ', '_BcWyKQL16ndpBdggh2kNA'), Confidence: 1.00000
 ('LhB4pLbijdV5nrSKN6pbIQ', '503VvfDu_HDvgUviAqmTEA'), Confidence: 1.00000

Top Frequent Triplets with Confidence:

('JITt4DBC550bSK3rNB1EOQ', '60RKNAZwY5EUooJf624aKw', 'lMY8NBPyzlPbbu-KBYfD9A'),
 Confidence: 1.00000
 ('WfVSrgBDH811lDU1nOg3Wg', 'Xw7ZjaGfr0WNVt6s_5KZfA', 'qpFCY_wj8_G-HOV8MxIaHw'),
 Confidence: 1.00000
 ('p_VpjA4PgghNylmXqDcnw', '_BcWyKQL16ndpBdggh2kNA', 'bJ5FtCtZX3ZZacz2_2PJjA'),
 Confidence: 1.00000
 ('p_VpjA4PgghNylmXqDcnw', 'ZCVEZmTB_StK-UXA3WPVlQ', '_BcWyKQL16ndpBdggh2kNA'),
 Confidence: 1.00000
 ('p_VpjA4PgghNylmXqDcnw', 'ZCVEZmTB_StK-UXA3WPVlQ', 'bJ5FtCtZX3ZZacz2_2PJjA'),
 Confidence: 1.00000
 ('G6uiBIcMc21mxtIsR6K_3w', 'VL12EhEdT40WqGq0nIqkzw', 'qjfMBIZpQT9DDtw_BWCopQ'),
 Confidence: 1.00000
 ('G6uiBIcMc21mxtIsR6K_3w', 'MmEYowrsA4j3K4gc9Th53g', 'Zn8tgekH0oCfyk4SUtygoA'),
 Confidence: 1.00000
 ('G6uiBIcMc21mxtIsR6K_3w', 'VL12EhEdT40WqGq0nIqkzw', 'Zn8tgekH0oCfyk4SUtygoA'),
 Confidence: 1.00000
 ('G6uiBIcMc21mxtIsR6K_3w', 'MmEYowrsA4j3K4gc9Th53g', 'VL12EhEdT40WqGq0nIqkzw'),
 Confidence: 1.00000
 ('G6uiBIcMc21mxtIsR6K_3w', 'MmEYowrsA4j3K4gc9Th53g', 'qjfMBIZpQT9DDtw_BWCopQ'),
 Confidence: 1.00000
 ('G6uiBIcMc21mxtIsR6K_3w', 'Zn8tgekH0oCfyk4SUtygoA', 'qjfMBIZpQT9DDtw_BWCopQ'),
 Confidence: 1.00000
 ('EkIIFm2zmql0YWXRHbwwqw', 'E4BsVQnG5zetbwv2x8QIWg', 'riWDgbauId8TK7YFVgMNJw'),
 Confidence: 0.91667
 ('WfVSrgBDH811lDU1nOg3Wg', 'Xw7ZjaGfr0WNVt6s_5KZfA', 'riWDgbauId8TK7YFVgMNJw'),
 Confidence: 0.90909
 ('WfVSrgBDH811lDU1nOg3Wg', 'qpFCY_wj8_G-HOV8MxIaHw', 'riWDgbauId8TK7YFVgMNJw'),
 Confidence: 0.90909
 ('rJZuQtzwh4S_XTMgh0ZG3w', '_BcWyKQL16ndpBdggh2kNA', 'bJ5FtCtZX3ZZacz2_2PJjA'),
 Confidence: 0.90000